

LLM Gateway Architecture

How production AI systems manage models, traffic, and reliability.



Routing



Rate Limits



Monitoring



Costs

What is an LLM Gateway?

PROXY

A layer between your application and LLM providers.

INSTEAD OF CALLING DIRECTLY:

App → OpenAI API

YOU USE A GATEWAY:

App → LLM Gateway → Model Providers

THE GATEWAY MANAGES



Request Routing



Rate Limiting



Model Selection



Monitoring



Fallbacks

MENTAL MODEL

API gateway for AI models

One Kit to Clear Your AI Interview

KIT

After seeing how AI interviews are evolving, I created an **AI Interview Kit**. It covers the areas companies actually test:



AI fundamentals



AI agents



Transformers (in depth)



Memory systems



LLM architectures



Vector databases



Prompt engineering



Backend for AI systems



Fine-tuning



Deployment strategies



RAG systems



System design for GenAI

Everything structured to help you prepare.

↓ [Check the caption to enroll!](#)

Why LLM Gateways Exist

INFRASTRUCTURE

Production AI systems face challenges like unpredictable latency, API limits, and rising token costs.

WITHOUT A GATEWAY

App → single model → failure risk

WITH A GATEWAY

App → Gateway → multiple models

THE GATEWAY HANDLES:

📌 Traffic management & resilience

Request Routing

SMART DISPATCH

WHAT IT IS

The gateway decides **which model should handle a request**.

EXAMPLE ROUTING LOGIC

```
def route_request(query):  
    if len(query) < 50:  
        return "gpt-3.5"  
  
    return "gpt-4"
```

THIS ALLOWS:

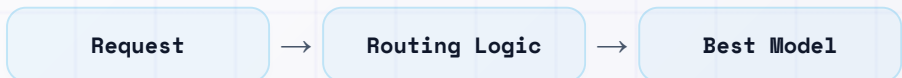


Cheaper models for simple tasks



Powerful models for complex tasks

MENTAL MODEL



Model Fallback

RELIABILITY

If the primary model fails, the gateway automatically switches to another.

EXAMPLE FALLBACK CHAIN

Primary model: **GPT-4**
Fallback model: **Claude**
Backup model: **Mistral**

EXAMPLE LOGIC

```
try:  
    response = call_gpt4(prompt)  
except:  
    response = call_claude(prompt) # Seamless
```

This ensures high availability.

MENTAL MODEL

Model Failure



Fallback Model

Rate Limiting

CONTROL

LLM APIs have strict usage limits. Gateways control request flow to avoid global bans.

EXAMPLE LOGIC

```
if requests_per_minute > limit:  
    reject_request()
```

BENEFITS



Prevents API abuse



Protects system stability



Controls usage spikes

MENTAL MODEL

Too Many Requests



Throttle Traffic

Cost Control

BUDGET

LLM usage can become expensive. A gateway helps manage and optimize costs.

EXAMPLE STRATEGY

Simple queries → Small model
Complex queries → Large model

EXAMPLE LOGIC

```
if task == "summarization":  
    model = "gpt-3.5"  
else:  
    model = "gpt-4"
```

This reduces unnecessary spending.

MENTAL MODEL

Task Complexity**Choose Model Cost**

Monitoring

TELEMETRY

Gateways track system performance across all underlying models.

EXAMPLE MONITORING OUTPUT:

GPT-4 latency: 2.4s
Claude latency: 1.9s
Error rate: 1.2%

CORE METRICS



Latency



Token Usage



Error Rates



Request Volume

MENTAL MODEL

All Requests



Tracked Centrally

Observability & Logging

AUDIT

Every single request and response can be logged for future review.

EXAMPLE LOG ENTRY

```
# Storing the entire interaction cycle
log_request(prompt, model, tokens, latency)
```

CAPTURED DIMENSIONS



User Request



Selected Model



Tokens Used



Latency



Raw Response

THIS HELPS:



debugging, auditing, & system improvement

Prompt Management

VERSIONING

Gateways can store and version prompts entirely apart from your application code.

EXAMPLE WORKFLOW

Prompt v1 → old system prompt
Prompt v2 → updated system prompt

EXAMPLE CODE USAGE

```
# Pulling prompts dynamically from gateway DB  
prompt = prompt_store.get("summarization_v2")
```

BENEFITS



Consistent prompts across clients



Easier updates without code deployments



A/B prompt experimentation

Safety & Guardrails

FILTER

The gateway can filter unsafe requests before they ever hit the expensive LLM layer.

EXAMPLE LOGIC

```
if contains_sensitive_data(prompt):  
    reject_request()
```

GATEWAY SAFEGUARDS



Sanitize prompts



Block harmful queries



Enforce policies

MENTAL MODEL

Request



Safety Filter



Model

Caching

SPEED

Semantic caching checks if queries repeat frequently, bypassing the LLM entirely.

EXAMPLE OUTPUT CACHE

User: "What is RAG?"

INSTEAD OF CALLING THE MODEL AGAIN:

```
if prompt in cache:  
    return cached_response
```

BENEFITS



Faster responses (low latency)



Reduced costs

Multi-Provider Support

ORCHESTRATION

Gateways allow using multiple model providers via a single unified interface.

EXAMPLE ARCHITECTURE

Gateway

- | OpenAI
- | Anthropic
- | Mistral
- | Local LLM

BENEFITS



Vendor independence (no lock-in)



Redundant reliability



Performance optimization

Streaming Responses

UX

Some applications need streaming outputs (ChatGPT style token streaming) to feel responsive.

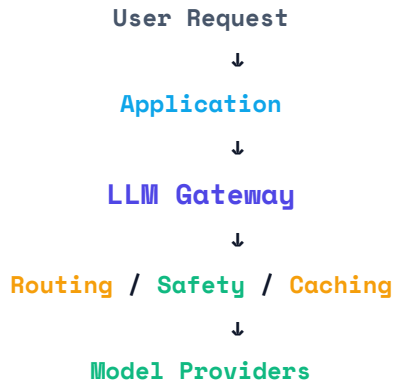
GATEWAY FORWARDING TOKENS AS THEY ARRIVE

```
for token in model.stream(prompt):  
    send_to_client(token)
```

This dramatically improves user experience by eliminating "dead waiting" time.

The Full LLM Gateway Architecture

SYSTEM



Serious GenAI systems are not just prompts + models.
They require **infrastructure**.

It's a core component of modern AI platforms.